VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELAGAVI – 590018

A PROJECT REPORT
on

"Two Way Communication System for Deaf and Dumb People"

Submitted in partial fulfillment of the requirement for the award of Degree of

Bachelor of Engineering In Computer Science and Engineering

Submitted by

SINCHANA	**4CA22CS099**
SOUNDARYA K S	**4CA22CS102**
THASHMITHA M H	**4CA22CS109**
VARSHINI M U	**4CA22CS113**

Under the Guidance of:

Prof. SAHANA G C
Assistant Professor

Department of Computer Science and Engineering

CIT, Mandya

Department of Computer Science and Engineering

Cauvery Institute of Technology

Siddaiahnakoppalu Gate, Sundahalli, Mandya – 571401

2025 – 2026

CAUVERY INSTITUTE OF TECHNOLOGY

SIDDAIAHNAKOPPALU GATE, SUNDAHALLI MANDYA-571401

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to Certify that the project entitled **"Two Way Communication System For Deaf and Dumb People"** is a bonafide work carried out by **SINCHANA** bearing USN **4CA22CS099** respectively in partial fulfilment for the degree of 7th Semester, Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belagavi during the year **2025-2026.** It is certified that all corrections or suggestions indicated for Internal Assessment have been incorporated on the report deposited in departmental library. The technical seminar report has been approved as it satisfies the academic requirements in respect of the technical seminar prescribed for the Bachelor of Engineering Degree.

---	---	---
Prof. Sahana G C	**Prof. Rakshitha B H**	**Dr. Srikantappa A S**
Asst. Professor	Asst. Professor & HOD	Principal
Dept. Of CSE	Dept. Of CSE	CIT, Mandya
CIT, Mandya	CIT, Mandya	

Name of the Examiners	**Signature with date**
1. _____	_____
2. _____	_____

ACKNOWLEDGMENT

Dreams never turn into reality unless a lot of efforts and hard work is put into it. It takes a lot of efforts to make your way to the goal and having someone to guide you and help you is always a blessing. So, we acknowledge all those whose guidance and encouragement served as a beacon light and crowned my efforts with success.

We thankful to **Dr. Srikantappa A S, Principal**, CIT, Mandya, for all the support he has rendered for providing me an opportunity to carry out our Major project.

We would like to thank **Prof. Rakshitha B H, Assistant Professor and Head of Department**, Department of Computer Science and Engineering, for her support for the successful completion of our major project.

We would like to sincerely thank our project co-ordinator**, Prof. Radhika K P, Assistant Professor,** Department of Computer Science & Engineering, CIT, Mandya, for her constant incitement and advice throughout the project.

We would like to sincerely thank our internal guide **Prof. Sahana G C, Assistant Professor,** Department of Computer Science and Engineering for providing relevant information, valuable guidance and encouragement to complete Major project.

SINCHANA

ABSTRACT

Communication is the foundation of human interaction, and language is the most powerful medium through which people express their thoughts, ideas, and emotions. However, for individuals who are deaf or non-speaking, conventional spoken language poses a significant barrier. Sign languages such as American Sign Language (ASL) and Indian Sign Language (ISL) serve as vital communication channels for these communities. Despite their importance, sign languages are not widely understood by the hearing population, which creates social and professional gaps. In today's digital era, technology provides opportunities to bridge this gap and create inclusive solutions for real-time communication. This project proposes a **Sign Language Translation System** designed to support both speech-to-sign and sign-to-text/voice translation. The first module captures live audio input, converts it into text using a speech recognition engine, and applies Natural Language Processing (NLP) techniques such as tokenization, stop-word removal, and part-of-speech tagging. The refined textual output is then mapped to a predefined dictionary of corresponding ASL gestures and displayed as animated visuals. This allows hearing individuals to communicate with deaf and non-speaking persons without requiring prior knowledge of sign language.The second module focuses on gesture recognition. Hand gestures are captured via a webcam, and hand landmarks are detected using MediaPipe. These landmarks are preprocessed and passed to an Artificial Neural Network (ANN) model trained to classify different ASL gestures. The system then translates the recognised gestures into text, enabling two-way communication. The combination of speech recognition, NLP, computer vision, and deep learning ensures that the system is capable of handling real-time communication tasks effectively.

CONTENTS

Acknowledgment	**I**
Abstract	**II**
Table Of Contents	**Page no**
Chapter 1 : Introduction	**1-5**
1.1 Introduction	1-2
1.2 Problem Statement	2
1.3 Objectives	2-3
1.4 Scope	3
1.5 Motivation	4
1.6 Existing System	4
1.7 Proposed System	4-5
Chapter 2 : Literature Survey	**6-9**
2.1 Speech-to-Sign Translation using NLP	6
2.2 MediaPipe for Gesture Recognition	6
2.3 Artificial Neural Networks for ASL Classification	6
2.4 Machine Learning Approaches in ISL Recognition	7
2.5 Deep Learning for Real-Time Sign Recognition	7
2.6 Hybrid Systems Combining Speech and Sign	7
2.7 Glove-Based Sign Recognition	8
2.8 Sign Language Animation for Communication	8-9
Chapter 3 : Software Requirement Specification (SRS)	**10-16**
3.1 Introduction to SRS	10
3.2 Scope	11
3.3 Definitions, Acronyms, and Abbreviations	11-13
3.4 Overview of Project	13-14
3.5 Functional Requirements	14-15
3.6 Non-Functional Requirements	15
3.7 Feasibility Study	15-16
Chapter 4 : System Design	**17-23**
4.1 Introduction to System Design	17
4.2 Architecture Diagram	17-19
4.3 DFD Level 1	19-20

4.4 Activity Diagram 20-21
4.5 Use Case Diagram 22
4.6 Sequence Diagram 23
Chapter 5 : Implementation 24-34
5.1 Introduction 24
5.2 Implementation with Respect to Our Project 25-26
5.3 Algorithmic Explanation 26-30
5.3.1 Hand Landmark Extraction (MediaPipe) 26
5.3.2 Landmark Preprocessing and Normalization 26-27
5.3.3 Dataset Creation and Logging 27-28
5.3.4 Gesture Classifier (ANN) 28
5.3.5 Post-Processing to Text 28-29
5.3.6 Audio Pipeline: Speech to ASL Tokenization 29
5.3.7 Animation Rendering 29
5.3.8 Text Polishing and Text-to-Speech 30
5.4 Pseudocode of Core Algorithms 30-34
5.4.1 Landmark Preprocessing 30
5.4.2 Dataset Logging 31
5.4.3 Real-Time Recognition Loop 31-32
5.4.4 Speech Text Normalization and Tense Heuristic 32-33
5.4.5 Token-to-Animation Mapping 33
5.4.6 Temporal Smoothing for Gesture Labels 34
Chapter 6 : Testing 35-40
6.1 Introduction to Testing 35
6.2 Types of Testing (in detail ≈1,000+ words) 36-40
6.2.1 Unit Testing 36
6.2.2 Integration Testing 36-37
6.2.3 System Testing 37-40
Chapter 7 : Result 41-42
7.1 Home Page 41
7.2 Convertor Page 41
7.3 Sign Up Page 42
7.4 Sequence Diagram 42

7.5 Contact Page 42
7.6 About Page 43
7.7 Text-to-Sign Translation 43
7.8 Sign-to-Text Translation 43
CONCLUSION **44**
FUTURE SCOPE **45**
REFERENCES **46-47**

LIST OF FIGURES

FIGURE NO	DESCRIPTION	Page no
4.2	Architecture Diagram	17-19
4.3	DFD Level 1	19-20
4.4	Activity Diagram	20-21
4.5	Use Case Diagram	22
4.6	Sequence Diagram	23
7.1	Home Page	41
7.2	Convertor Page	41
7.3	Sign Up Page	42
7.4	Sign Recognition	42
7.5	Contact Page	42
7.6	About Page	43
7.7	Text-to-Sign Translation	43
7.8	Sign-to-Text Translation	43

CHAPTER 1

INTRODUCTION

1.1 Introduction

Language is the backbone of human civilisation and is essential for sharing knowledge, building relationships, and promoting cultural growth. Spoken languages dominate as the primary form of interaction. However, for individuals who are deaf or non-speaking, spoken communication presents profound challenges. Sign languages, which are natural and expressive, have emerged as powerful tools for communication. American Sign Language (ASL) and Indian Sign Language (ISL) are widely used, but they are not universally understood by the hearing population. This lack of common understanding often results in social exclusion and communication barriers in education, healthcare, and workplaces.

Technology has the potential to act as a bridge between these two worlds. Advances in machine learning, natural language processing (NLP), and computer vision provide opportunities to develop intelligent systems that can translate between spoken language and sign language. Such systems can make communication more inclusive and help break down long-standing barriers.

This project introduces a **Sign Language Translation System** that operates in two directions. The first module captures audio input, processes it into text using speech recognition, and refines it through NLP techniques like tokenization and part-of-speech tagging. The processed text is then mapped to ASL gestures and displayed as animations. This enables hearing individuals to convey their message in sign language without having to learn it.

The second module works in the reverse direction. A webcam captures hand gestures, and MediaPipe is used to extract hand landmarks. These features are fed into an Artificial Neural Network (ANN) trained on different ASL gestures. The model outputs the recognised gestures, which are then translated into text. This empowers deaf and non-speaking individuals to express themselves clearly to those unfamiliar with sign language.

The integration of these two modules creates a comprehensive and interactive communication framework. The solution can be applied in multiple real-life contexts, such as classrooms where deaf students can interact with teachers, hospitals where patients can explain their needs, and workplaces where inclusive communication is vital. Additionally, the system

promotes awareness of sign languages among hearing individuals, encouraging inclusivity and acceptance.

In summary, this project is a step towards building an inclusive society where technology empowers everyone to communicate freely. It demonstrates the application of speech processing, NLP, and deep learning in creating impactful solutions for bridging human communication gaps.

1.2 Problem Statement

Deaf and non-speaking individuals face persistent communication barriers due to limited understanding of sign language among the hearing population. This leads to social isolation and restricted opportunities in education, healthcare, and workplaces. Existing tools either focus on speech-to-sign or sign-to-text but rarely provide a complete bidirectional solution. Therefore, there is a pressing need for a low-cost, real-time, and accessible system that can translate between spoken language and sign language seamlessly. The proposed project aims to design such a system by integrating speech recognition, NLP, computer vision, and machine learning techniques.

1.3 Objectives

The primary objective of this project is to design and implement a **bidirectional sign language translation system** that facilitates communication between hearing individuals and those who are deaf or non-speaking.

Specific objectives include:-

1. **Speech-to-Sign Translation**
    - Capture live audio input and convert it into text using speech recognition.
    - Refine the transcribed text using NLP techniques such as tokenization, stop-word removal, and part-of-speech tagging.
    - Map the processed text to a predefined dictionary of ASL gestures.
    - Display corresponding gestures through animated visuals for real-time communication.

2. **Sign-to-Text Translation**
   - Capture hand gestures using a webcam in real time.
   - Detect and extract hand landmarks using MediaPipe.
   - Preprocess and classify gestures using an ANN trained on ASL datasets.
   - Convert the recognised gestures into textual output for clear understanding.
3. **Bidirectional Integration**
   - Combine both modules into a single framework to enable two-way interaction.
   - Ensure the system operates in real time with minimal latency.
4. **Practical Applications**
   - Develop a user-friendly interface that can be applied in classrooms, healthcare, workplaces, and social platforms.
   - Promote inclusivity by enabling non-signers to communicate with the deaf community.

By fulfilling these objectives, the project seeks to reduce communication gaps, foster inclusivity, and demonstrate the role of technology in empowering marginalised communities.

1.4 Scope

The scope of this project is limited to the development of a prototype capable of translating basic sentences into ASL animations and recognising a predefined set of ASL gestures through ANN models. The system focuses on real-time performance and accuracy in controlled environments. While the dictionary may not cover the full grammar and richness of sign language, it provides a foundation for scalable solutions. Future extensions may include multilingual support, mobile integration, and expansion to Indian Sign Language (ISL). The system is intended for educational, healthcare, and social use cases where communication barriers are most critical.

1.5 Motivation

The motivation behind this project stems from the urgent need to create inclusive communication tools for the deaf and non-speaking community. In India and worldwide, millions face barriers in education, employment, and social participation due to a lack of sign language awareness. By leveraging recent advances in NLP and deep learning, it is possible to design systems that reduce these barriers. The project is driven by a vision of inclusivity, social equity, and empowerment, ensuring that no individual is left behind due to communication differences. This effort reflects the belief that technology should serve humanity as a whole.

1.6 Existing System

Current systems for sign language translation are either one-directional or highly constrained. Many solutions translate speech or text into sign language animations, but these systems often lack natural sign grammar or rely on limited dictionaries. On the other hand, gesture recognition systems using gloves, sensors, or webcams convert signs into text but usually support only a small vocabulary. Furthermore, most systems are not integrated into a unified framework for bidirectional communication. Their complexity, cost, or limited usability restricts real-world adoption. Thus, there exists a gap for a low-cost, user-friendly, real-time, and comprehensive translation system.

1.7 Proposed System

The proposed system introduces a **bidirectional sign language translation framework** that integrates both speech-to-sign and sign-to-text modules into one unified solution.

In the **speech-to-sign module**, live audio input is captured through a microphone. A speech recognition engine converts the spoken words into text. This raw text is processed through NLP steps like tokenization, stop-word removal, and part-of-speech tagging to refine the output. The processed text is then mapped to a predefined dictionary of ASL gestures, which are displayed as animations. This enables hearing individuals to communicate directly with deaf and non-speaking persons in sign language.

In the **sign-to-text module**, gestures are captured through a webcam. Using MediaPipe, the system extracts hand landmarks and converts them into numerical features. These features are preprocessed and fed into an Artificial Neural Network (ANN) trained on a dataset of ASL

gestures. The ANN predicts the gesture class, which is then mapped to its textual equivalent. This translation allows deaf and non-speaking individuals to express themselves clearly to those unfamiliar with sign language.

The novelty of the system lies in its bidirectional capability, real-time performance, and user-friendly design. It eliminates the need for specialised hardware such as gloves or sensors, relying only on accessible tools like microphones and webcams. This makes the system affordable and scalable for deployment in diverse settings.

CHAPTER 2

LITERATURE SURVEY

2.1 Speech-to-Sign Translation using NLP

A study by Kumar and Rani (2019) proposed a speech-to-sign translation model that combined automatic speech recognition with natural language processing. The system converted spoken English into text, applied tokenisation and part-of-speech tagging, and then mapped the words to Indian Sign Language (ISL) animations. Their work highlighted the importance of preprocessing steps like stop-word removal to ensure grammatical clarity. While effective for basic communication, the system struggled with complex sentence structures and lacked real-time responsiveness. Nevertheless, the research laid the groundwork for integrating NLP into sign language translation frameworks.

2.2 Media Pipe for Gesture Recognition

In 2020, Google introduced Media Pipe as a powerful tool for real-time hand tracking and landmark detection. Researchers such as Zhang et al. (2021) leveraged MediaPipe for recognising American Sign Language (ASL) gestures without the need for gloves or sensors. By extracting hand landmarks and feeding them into a neural network, their system achieved high accuracy in static gesture recognition. However, the study also noted challenges in recognising continuous gestures and two-handed signs. The use of MediaPipe demonstrated that low-cost, camera-based solutions can significantly enhance accessibility for sign language recognition systems.

2.3 Artificial Neural Networks for ASL Classification

Sharma and Gupta (2020) experimented with artificial neural networks (ANNs) to classify ASL gestures. Their dataset consisted of hand images labelled with corresponding gestures. After preprocessing, features were extracted and classified using a feedforward neural network. The system achieved promising results for alphabets and basic words. The study emphasised that ANNs could learn gesture patterns effectively, but their performance was highly dependent on the dataset's diversity. The work concluded that combining ANNs with advanced techniques like CNNs or RNNs could further improve recognition of dynamic gestures.

2.4 Machine Learning Approaches in ISL Recognition

A research work by Singh et al. (2018) developed an Indian Sign Language recognition model using Support Vector Machines (SVM). The system processed hand gesture images, extracted shape-based features, and classified them into different ISL signs. Their results showed good accuracy for a small dataset of isolated gestures. However, the authors admitted that SVM lacked scalability for large datasets and continuous sign recognition. This study is significant as it demonstrated the early use of machine learning for ISL, paving the way for later deep learning-based approaches.

2.5 Deep Learning for Real-Time Sign Recognition

Rahman et al. (2021) explored convolutional neural networks (CNNs) for real-time sign recognition using webcams. Their system captured video frames, detected hand regions, and classified gestures into ASL alphabets. The CNN model achieved higher accuracy compared to traditional machine learning methods. Importantly, the study addressed challenges like varying lighting conditions and background noise. Although limited to alphabets, the research proved that deep learning could handle real-time translation tasks effectively and could be scaled to larger vocabularies with further training.

2.6 Hybrid Systems Combining Speech and Sign

In 2020, Patel and Mehta designed a hybrid communication system that supported both speech-to-sign and sign-to-speech translation. The speech module converted audio into text, which was then mapped to sign animations. The sign module captured gestures using a depth camera and translated them into speech output. Their system demonstrated the feasibility of bidirectional translation, although the hardware requirements made it less accessible for general users. The study remains important as it emphasised the need for integrated systems that serve both hearing and deaf communities.

2.7 Glove-Based Sign Recognition

Earlier works, such as by Joshi et al. (2016), focused on glove-based systems for recognising sign gestures. These gloves were embedded with sensors that captured finger bending and motion, which were then processed by a computer. The advantage of this approach was high precision in gesture recognition. However, the system was costly, uncomfortable, and not practical for daily use. Although outdated compared to camera-based approaches, glove-based systems contributed to early progress in sign language recognition research.

2.8 Sign Language Animation for Communication

Research by Li and Chen (2017) focused on developing animated avatars to display sign language gestures. Instead of using videos, the system generated 3D animations of signs based on linguistic rules. This approach made it easier to generate scalable sign content without recording thousands of human signers. However, the animations lacked natural facial expressions and body movements, which are crucial in sign language grammar. Despite its limitations, this study introduced the idea of using animated visuals as a cost-effective means to bridge communication between hearing and deaf individuals.

Author & Year	**Method / Approach**	**Strengths**	**Limitations**
Kumar & Rani (2019)	Speech-to-sign using NLP (speech recognition + POS tagging + mapping to ISL)	Simple pipeline, good for basic phrases	Struggles with complex sentences, limited vocabulary
Zhang et al. (2021)	MediaPipe for gesture recognition with ANN	Real-time hand tracking without sensors, cost-effective	Difficulties with continuous and two-handed signs
Sharma & Gupta (2020)	Artificial Neural Networks for ASL gesture classification	High accuracy for static alphabets, ANN learns patterns well	Dataset-dependent, poor for dynamic gestures
Singh et al. (2018)	ISL recognition using Support Vector Machines (SVM)	Effective for small isolated sign datasets	Not scalable, struggles with continuous gestures
Rahman et al. (2021)	Deep learning (CNNs) for real-time ASL recognition	Robust to lighting/background noise, better accuracy than ML	Limited to alphabets, needs larger datasets
Patel & Mehta (2020)	Hybrid system: Speech-to-sign + Sign-to-speech	Bidirectional communication, proof of concept	Requires depth cameras, not accessible for daily use
Joshi et al. (2016)	Glove-based sign recognition (sensor gloves)	High precision in capturing finger movement	Expensive, impractical, uncomfortable for users
Li & Chen (2017)	3D animated avatars for sign language output	Scalable, avoids video recording, low-cost visualisation	Lacks natural facial expressions and grammar elements

CHAPTER 3

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

3.1 Introduction to SRS

A Software Requirement Specification (SRS) is a structured document that clearly defines the functional and non-functional requirements of a system under development. It serves as a bridge between the client's needs and the developer's technical implementation. The primary objective of an SRS is to provide a comprehensive understanding of what the system should do, how it should behave, and the constraints under which it must operate.

In the context of this project—**Sign Language Translation System for Deaf and Non-Speaking Individuals**—the SRS acts as the foundation for both design and development. Since the system integrates speech recognition, natural language processing (NLP), computer vision, and artificial intelligence techniques, it is necessary to describe the requirements in precise detail to ensure clarity and prevent misinterpretation.

The SRS describes the purpose of the system, the scope of its application, the functional and non-functional requirements, the project feasibility, and the constraints. It also defines specific terminologies, acronyms, and abbreviations used throughout the project. By doing so, it ensures that both technical and non-technical stakeholders—developers, testers, project managers, domain experts, and end users—can understand the system requirements.

Furthermore, the SRS acts as a reference document throughout the development lifecycle. It reduces ambiguity, facilitates effective communication, and provides a contractual agreement between stakeholders. In software engineering practice, a well-documented SRS ensures that the system is developed in alignment with user needs, within the specified time frame and budget.

Thus, the SRS for the Sign Language Translation System forms the cornerstone of this project, outlining not only the features but also the quality benchmarks and feasibility considerations necessary to build a robust, user-friendly, and inclusive solution.

3.2 Scope

The scope of the Sign Language Translation System covers the development of a **bidirectional communication platform** enabling interaction between hearing individuals and deaf/non-speaking persons. The system translates spoken input into animated sign language gestures and recognises sign language gestures to produce textual output. The solution employs speech recognition, NLP, and ANN-based gesture recognition integrated into a single framework. The project is limited to recognising a predefined vocabulary of ASL gestures and simple sentence structures. While scalability to Indian Sign Language (ISL) and mobile platforms is possible, this version focuses on a prototype applicable in educational, healthcare, and workplace environments.

3.3 Definitions, Acronyms, and Abbreviations

This section defines key technical terms, acronyms, and abbreviations relevant to the project. Providing these definitions ensures a shared understanding of the terminology used in subsequent sections.

General Terms

- **Sign Language**: A natural visual-manual language using hand gestures, facial expressions, and body movements to convey meaning. Examples include American Sign Language (ASL) and Indian Sign Language (ISL).
- **Deaf**: Refers to individuals with significant hearing loss, who may rely on sign language as their primary mode of communication.
- **Non-Speaking**: Individuals who cannot produce spoken language due to medical, developmental, or physical conditions, but who may use alternative modes of communication such as sign language.
- **Gesture Recognition**: A process of interpreting human gestures using computational algorithms, often involving image processing and machine learning.
- **Bidirectional Communication**: Two-way interaction between two parties, in this case between hearing individuals and deaf/non-speaking individuals.

Technical Terms

- **Speech Recognition**: Technology that converts spoken words into text using acoustic and language models.
- **Natural Language Processing (NLP)**: A field of artificial intelligence concerned with the interaction between computers and human languages, including tasks such as tokenisation, part-of-speech tagging, and stop-word removal.
- **Tokenisation**: Breaking text into smaller units such as words or phrases.
- **Stop-Word Removal**: Eliminating commonly used words (e.g., “the”, “is”, “and”) that do not add semantic value to processing.
- **Part-of-Speech Tagging (POS Tagging)**: Identifying grammatical categories (e.g., noun, verb, adjective) of words in a sentence.
- **Dictionary Mapping**: Linking recognised words to predefined sign language gestures in a database.
- **Animation Module**: A software component that visualises gestures using 2D/3D animated characters.

Artificial Intelligence and Machine Learning Terms

- **Artificial Neural Network (ANN)**: A machine learning model inspired by the structure of the human brain, used for classifying ASL gestures in this project.
- **Convolutional Neural Network (CNN)**: A type of ANN particularly effective in image and gesture recognition tasks.
- **Training Dataset**: A collection of labelled images or sequences used to train machine learning models.
- **Testing Dataset**: A dataset used to evaluate the performance of trained models.
- **Accuracy**: A measure of how many predictions made by a model are correct.
- **Overfitting**: A condition where a model learns training data too well but fails to generalise to new data.

Frameworks and Tools

- **MediaPipe**: A cross-platform framework developed by Google for building real-time perception pipelines, used here for hand landmark detection.
- **TensorFlow**: An open-source machine learning framework used to build and train neural network models.
- **OpenCV**: A computer vision library that assists in image and video processing tasks.
- **PyTorch**: An alternative deep learning framework, widely used for research and deployment.

Acronyms

- **ASL**: American Sign Language
- **ISL**: Indian Sign Language
- **NLP**: Natural Language Processing
- **POS**: Part of Speech
- **ANN**: Artificial Neural Network
- **CNN**: Convolutional Neural Network
- **API**: Application Programming Interface
- **UI/UX**: User Interface / User Experience
- **JSON**: JavaScript Object Notation, used for data exchange

3.4 Overview of Project

The proposed Sign Language Translation System is designed to enable seamless communication between hearing individuals and those who are deaf or non-speaking. The project integrates two modules: **speech-to-sign translation** and **sign-to-text translation**.

The first module captures spoken input through a microphone and converts it into text using a speech recognition engine. NLP techniques are applied to refine the text by removing unnecessary words and tagging grammatical structures. The refined text is then mapped to a gesture dictionary, and the output is displayed as sign animations. This enables non-signers to communicate their thoughts in sign language without prior knowledge of ASL.

The second module captures real-time gestures through a webcam. Using MediaPipe, hand landmarks are extracted, which are then preprocessed and classified by an ANN model. The recognised gestures are converted into textual equivalents and displayed on the screen, enabling deaf and non-speaking users to communicate effectively with hearing individuals.

Together, these two modules form a **bidirectional communication system**. The project emphasises affordability and accessibility, using webcams and microphones instead of expensive hardware like gloves or sensors. It also focuses on scalability, with possibilities for integration into mobile platforms and support for multiple sign languages in future iterations.

Applications include classrooms, hospitals, workplaces, and social contexts where inclusivity is essential. The system not only empowers deaf and non-speaking individuals but also raises awareness of sign languages among the general population.

3.5 Functional Requirements

Functional requirements describe what the system should do. For this project, they include:

1. **Speech Input Processing**
   - Capture live audio input.
   - Convert audio into text using a speech recognition engine.
   - Refine text with NLP techniques (tokenisation, stop-word removal, POS tagging).
2. **Sign Animation Output**
   - Map refined text to predefined ASL gestures.
   - Display gestures as animated visuals in real time.
3. **Gesture Recognition**
   - Capture gestures using a webcam.
   - Detect hand landmarks with Media Pipe.
   - Preprocess and classify gestures using ANN.
4. **Text Output Generation**
   - Convert classified gestures into text.

- Display the output text on screen.

5. **User Interface**
    - Provide a simple, intuitive GUI for input and output.
    - Allow switching between speech-to-sign and sign-to-text modules.

3.6 Non-Functional Requirements

Non-functional requirements specify how the system should perform:

- **Performance**: The system must operate in real time with minimal latency.
- **Accuracy**: Speech recognition and gesture classification should achieve at least 85% accuracy in controlled environments.
- **Scalability**: The system should be extendable to larger vocabularies and additional sign languages.
- **Usability**: The interface should be simple and user-friendly for non-technical users.
- **Portability**: The software should run on common operating systems (Windows/Linux).
- **Reliability**: The system should remain stable under continuous operation.
- **Security**: User data, especially video and audio, must be handled securely without unauthorised access.
- **Maintainability**: The codebase should be modular to allow updates.

3.7 Feasibility Study

1. Technical Feasibility

The system uses widely available tools like Media Pipe, TensorFlow, and Python libraries, making it technically feasible. Affordable hardware such as webcams and microphones ensures accessibility.

2. Operational Feasibility

Users can operate the system with minimal training. A user-friendly GUI ensures accessibility for both technical and non-technical individuals.

3. Economic Feasibility

The system avoids costly equipment like sensor gloves, relying instead on open-source frameworks. This makes it affordable for deployment in schools, hospitals, and small organisations.

4. Legal and Ethical Feasibility

The system complies with ethical considerations by promoting inclusivity and accessibility. However, datasets must be licensed properly to avoid copyright issues.

5. Schedule Feasibility

Given the modular approach, the system can be developed within a semester-long project timeline.

CHAPTER 4

SYSTEM DESIGN

4.1 Introduction to System Design

System Design is the process of translating the Software Requirement Specification (SRS) into a structured solution that outlines how the system will be implemented. It involves defining the system's architecture, modules, data flow, interactions, and overall behaviour. While the SRS answers the question of *what* the system should do, the System Design chapter answers the question of *how* the system will do it.

In this project, the Sign Language Translation System integrates multiple technologies such as speech recognition, Natural Language Processing (NLP), Media Pipe-based hand tracking, and Artificial Neural Networks (ANN). Therefore, careful design is essential to ensure smooth integration of these modules into a unified framework.

The design process follows both top-down and modular approaches. First, the system is viewed as a whole to establish its architecture, followed by decomposition into smaller components such as speech-to-sign, sign-to-text, and user interface. Each component is then modelled through diagrams like Data Flow Diagrams (DFD), Activity Diagrams, Use Case Diagrams, and Sequence Diagrams.

The purpose of system design is to:

- Define the architecture of the system in terms of components and their interactions.
- Provide clear visual models that help developers and stakeholders understand the functionality.
- Minimise risks of integration issues by identifying dependencies early.
- Ensure that the design aligns with functional and non-functional requirements.

Thus, system design forms the blueprint for implementation, ensuring the project is both robust and user-friendly.

4.2 Architecture Diagram

The architecture of the Sign Language Translation System is **modular and layered**, comprising two main functional modules: **Speech-to-Sign Translation** and **Sign-to-Text**

Translation. Both modules interact through a central **Application Layer** that controls data flow and manages outputs.

1. **Input Layer**
   - For the speech module, input is captured using a microphone.
   - For the sign recognition module, a webcam captures video input of hand gestures.
2. **Processing Layer**
   - **Speech-to-Sign**: Audio is processed by a Speech Recognition Engine (e.g., Google Speech API). The transcribed text undergoes NLP techniques like tokenisation, stop-word removal, and part-of-speech tagging. The processed text is mapped to predefined gestures.
   - **Sign-to-Text**: MediaPipe detects hand landmarks. These numerical features are preprocessed and fed into an Artificial Neural Network (ANN) trained to recognise ASL gestures. The model outputs the recognised gesture class.
3. **Application Layer**
   - Controls the mapping between processed inputs and their outputs.
   - Ensures real-time communication between the two modules.
4. **Output Layer**
   - For the speech module, the mapped gestures are displayed as animated visuals.
   - For the sign module, the recognised gestures are displayed as text.

This architecture ensures a **bidirectional communication system**. The modularity allows independent improvements in speech processing or gesture recognition without disturbing the rest of the system.

In short, the architecture combines speech recognition, NLP, computer vision, and machine learning into a cohesive design. It is both **scalable** (future addition of Indian Sign Language support) and **affordable** (no special hardware required apart from webcam and microphone).

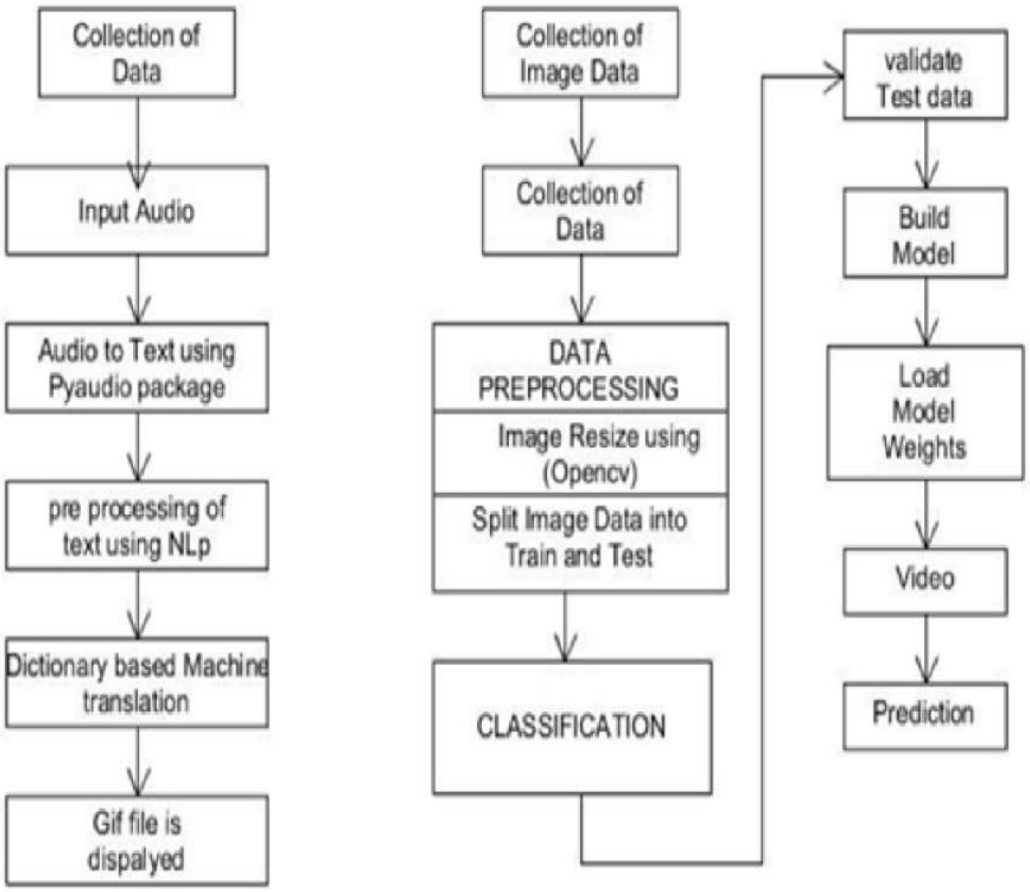
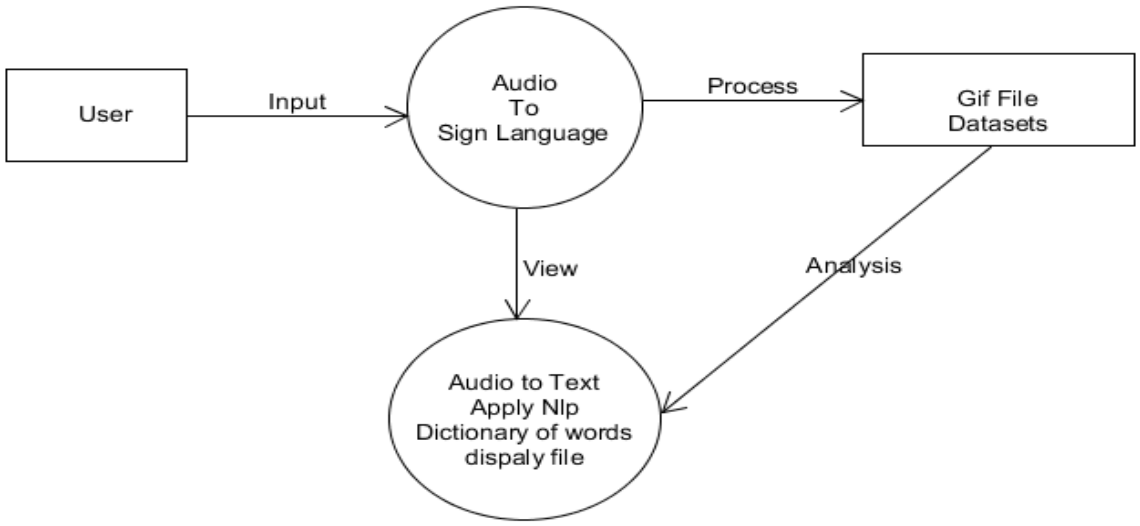


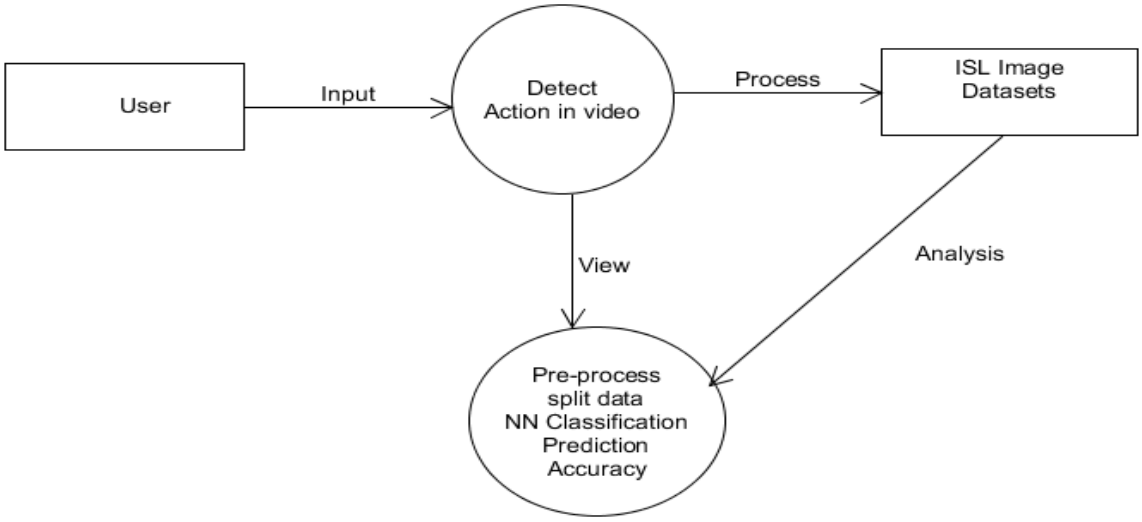
Figure 4.2 Architecture Diagram

4.3 DFD Level 1

In the Level 1 DFD, the main process is broken into two sub-processes: **Speech-to-Sign** and **Sign-to-Text**. The Speech-to-Sign sub-process receives audio, converts it into text, applies NLP, and maps it to gestures that are shown as animations. The Sign-to-Text sub-process captures gestures via webcam, detects hand landmarks using MediaPipe, and classifies them using ANN before displaying the corresponding text. Both sub-processes interact with a **Gesture Dictionary Database** that stores mappings of words and gestures. This DFD provides clarity on internal data flow and module responsibilities.



(a) Speech-to-Sign



(b) Sign-to-Text

Figure 4.3 DFD Level 1

4.4 Activity Diagram

The Activity Diagram shows the sequence of actions from user input to system output. For the Speech-to-Sign module, the flow begins with **speech capture**, followed by **speech recognition**, **NLP processing**, **dictionary mapping**, and finally **gesture animation output**. For the Sign-to-Text module, the flow begins with **gesture capture**, **hand landmark detection**, **ANN classification**, and **text output generation**. Decision points such as selecting the module (speech-to-sign or sign-to-text) are represented, ensuring clarity of

control flow. This diagram explains how user activities are executed in a step-by-step manner within the system.

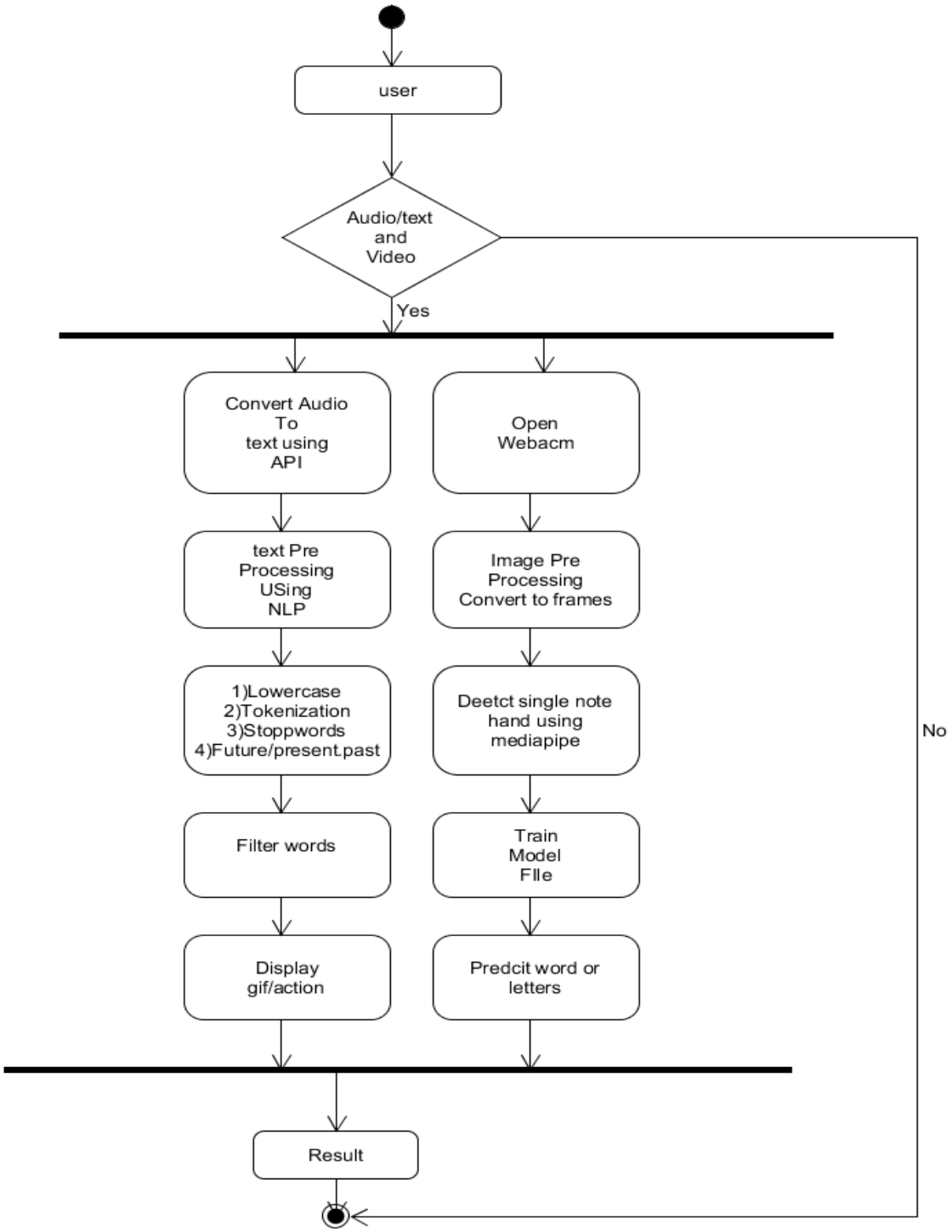


Figure 4.4 Activity Diagram

4.5 Use Case Diagram

The Use Case Diagram identifies the main actors and their interactions with the system. The two primary actors are **Hearing Individual** and **Deaf/Non-Speaking Individual**. The Hearing Individual uses the **Speech-to-Sign Translation Use Case**, which involves speaking into the system and receiving sign animations. The Deaf/Non-Speaking Individual uses the **Sign-to-Text Translation Use Case**, which involves performing gestures and receiving textual translations. Both use cases interact with the system's internal modules, ensuring bidirectional communication. The diagram clearly represents the scope of user-system interaction and demonstrates the inclusivity achieved by the project.

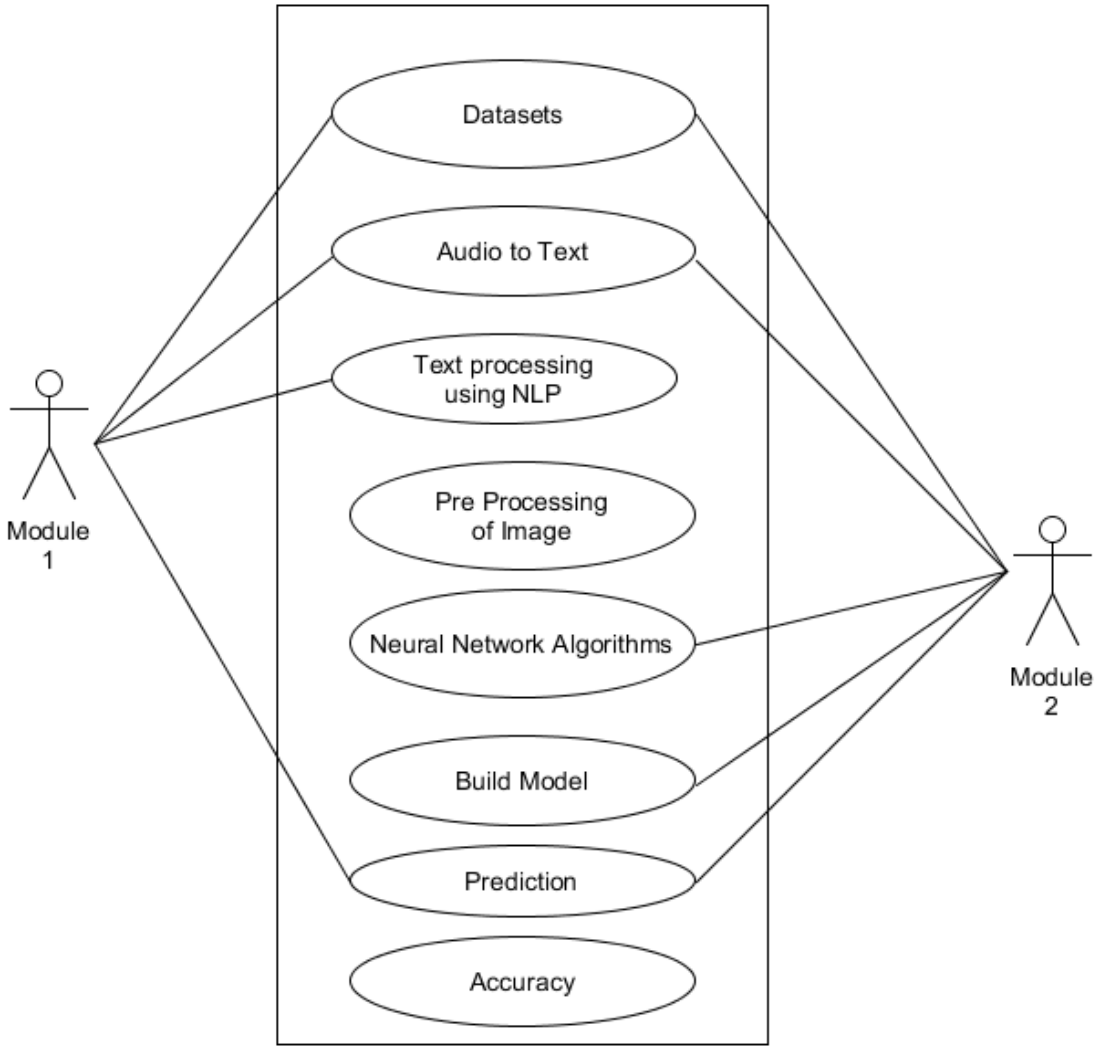


Figure 4.5 Use Case Diagram

4.6 Sequence Diagram

The Sequence Diagram depicts the chronological order of operations. For the Speech-to-Sign process, the **User** sends speech input to the **System**, which calls the **Speech Recognition Module**, then the **NLP Module**, and finally the **Gesture Animation Module** before returning sign output. For the Sign-to-Text process, the **User** performs gestures captured by the **Camera Module**, passed to the **Gesture Recognition Module**, classified by the **ANN Model**, and returned as text. This sequence shows how different modules collaborate in real time, ensuring smooth communication between the two user groups.

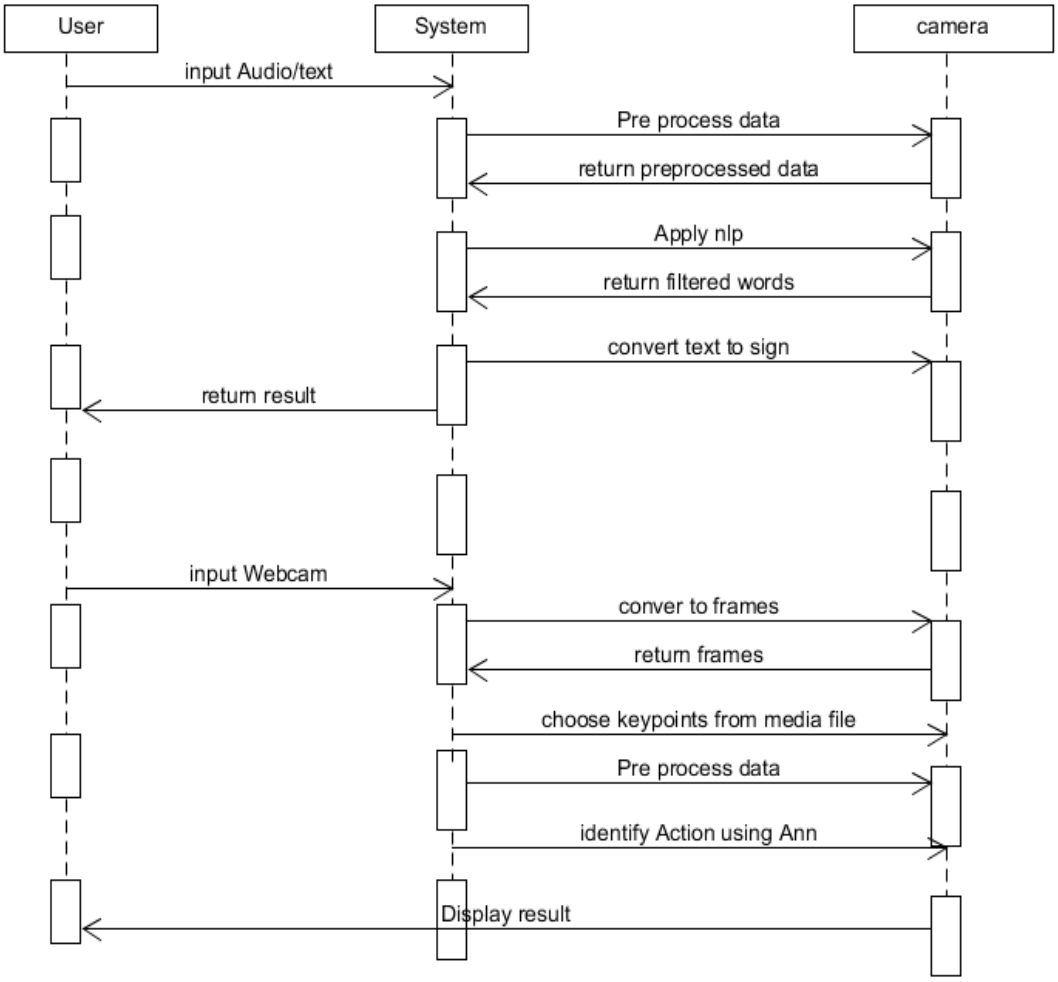


Figure 4.6 Sequence Diagram

CHAPTER 5

IMPLEMENTATION

5.1 Introduction

This chapter details how the Smart Audio Sign Language (SASL) system was engineered from concept to a working, deployable application. Whereas earlier chapters motivated the need and surveyed related work, this chapter narrows the focus to the concrete decisions that make the system function in real time on commodity hardware. The implementation blends three families of techniques: (i) speech understanding and natural language processing (NLP) for converting spoken English into a linguistically tidy, ASL-oriented token stream; (ii) computer-vision–based gesture recognition for mapping live hand poses to symbolic ASL units; and (iii) a lightweight web stack for delivering a responsive user interface and packaging the models for edge deployment.

On the audio→sign pathway, the pipeline captures microphone input, performs speech-to-text, cleans the transcription with standard NLP primitives (tokenization, stop-word removal, part-of-speech tagging, and lemmatization), and then maps the resulting tokens to sign units. When a full-word animation is unavailable, the system gracefully degrades to letter-by-letter fingerspelling. On the sign→text pathway, a webcam stream is analyzed by MediaPipe Hands to produce 2D hand landmarks. After geometric normalization, these features are passed to a compact artificial neural network (ANN) classifier that predicts gesture classes. A text-level post-processor smooths the output into grammatical English and optionally renders it as speech.

The application is delivered through Django. The server orchestrates the two pathways, serves the animation assets, and exposes a clean interface for recognition and animation playback. Performance considerations—frame-rate stability, latency budgets, and robustness to lighting and scale—guided numerous design choices, including precomputing animation lookups, vectorized preprocessing, and converting the trained ANN into TensorFlow Lite (TFLite) for accelerated inference on CPUs such as the Raspberry Pi.

What follows is an end-to-end description of the modules, their data flows, the algorithms they implement, and the rationale for the parameters used. Where appropriate, complexity analyses and implementation tips are included to inform replication and future extensions.

5.2 Implementation with Respect to Our Project

The project is implemented as two cooperating modules exposed through a Django web interface:

1. **Audio-to-Sign Module**
   - **Input**: Live audio captured through the browser or system microphone.
   - **Processing**: Speech recognition converts audio to text; NLP routines normalize the text by removing low-information words, tagging parts of speech, and lemmatizing inflected forms. A tense-aware heuristic inserts temporal markers ("Before," "Will," "Now") so that ASL users receive temporal context often expressed non-verbally in ASL.
   - **Output**: A sequence of sign units. For each token, the view attempts to load a pre-authored animation clip (e.g., token.mp4). If the clip is absent, the token is decomposed into characters and fingerspelling clips are queued. The animation view (animation_view) renders this sequence for the user.
2. **Sign-to-Text Module**
   - **Input**: Webcam frames acquired at 30+ FPS.
   - **Processing**: MediaPipe Hands finds 21 landmarks per detected hand. Landmarks are converted to a camera-invariant, scale-normalized coordinate system and passed to a trained ANN classifier (KeyPointClassifier).
   - **Output**: Gesture labels mapped to English words. A lightweight grammar smoother (TextBlob, plus simple tense handling) produces readable text, which can be spoken using a text-to-speech engine.

Two helper programs support data creation and runtime inference:

- **preparedatasets.py** captures landmark sequences, labels them via keyboard shortcuts (0–9), and writes normalized vectors to CSV, forming the training set.
- **main.py** handles the real-time recognition loop: video capture, landmark extraction, preprocessing, classification, and overlay of predictions.

For deployment on constrained hardware, the trained ANN is quantized and exported to **TFLite**, dramatically reducing memory footprint and improving inference time without notable accuracy loss. The result is a responsive, low-cost, and accessible tool that operates smoothly on laptops and single-board computers alike.

5.3 Algorithmic Explanation

This section explains the principal algorithms and engineering choices behind SASL, organized by pipeline stage.

5.3.1 Hand Landmark Extraction (MediaPipe)

Goal. Convert each RGB frame into a compact, informative representation of hand pose.

Method. MediaPipe Hands uses a palm detector followed by a landmark regressor to return 21 normalized 2D landmarks per hand. For each hand, we obtain coordinates $(x_i, y_i)$for $i \in \{0, \dots, 20\}$. MediaPipe also returns handedness and a detection confidence score. In our application, we set:

- max_num_hands = 2 to recognize both hands when available;
- min_detection_confidence and min_tracking_confidence tuned to stabilize landmark flicker while avoiding false positives in cluttered backgrounds.

Why MediaPipe? It is robust to rotation, partial occlusion, and lighting changes. Most importantly, it is fast: it runs in real time on CPU, which aligns with our edge-friendly deployment target.

Complexity. Landmark extraction is roughly $O(1)$per frame with a fixed-size neural network. Latency is dominated by the neural runtime and image I/O.

5.3.2 Landmark Preprocessing and Normalization

Raw landmarks depend on camera position, user distance, and hand size. To make classification invariant to these nuisance factors, we normalize as follows:

1. **Reference-Point Translation.** Subtract the wrist landmark $(x_0, y_0)$from all points: $(x_i', y_i') = (x_i - x_0, y_i - y_0)$.
   This centers the hand at the origin.

2. **Scale Normalization.** Compute a characteristic scale s. Options include the maximum absolute coordinate $\max_i\{|x_i'|, |y_i'|\}$or the distance between a pair of anatomically stable landmarks (e.g., wrist to middle fingertip). We divide all coordinates by $s + \epsilon$. This enforces size invariance.

3. **Optional Orientation Normalization.** To reduce rotation variance, we can rotate the point cloud so that the wrist–index metacarpophalangeal (MCP) vector aligns with a canonical axis. In our minimal implementation, we rely on the ANN to learn rotational tolerance; however, adding explicit rotation normalization can help with small datasets.

4. **Flattening.** The normalized coordinates are flattened into a fixed-length vector $\mathbf{z} \in \mathbb{R}^{42}$(two values per landmark × 21 landmarks).

This preprocessing is implemented in pre_process_landmark and applied consistently in both preparedatasets.py (for training data) and main.py (for inference), ensuring that the model sees identically normalized features in both phases.

Robustness Considerations.

- **Noise Resistance:** Translating and scaling greatly reduce intra-class variance.
- **Two-Hand Cases:** If two hands are present, we classify each separately and then apply a fusion rule if the current vocabulary includes two-hand signs (future work can extend to explicit bimanual patterns).

5.3.3 Dataset Creation and Logging

Accurate models require curated data. Our dataset tool (preparedatasets.py) captures landmark vectors and writes them to CSV along with a label:

- The user holds a sign; the script displays the live feed with drawn landmarks.
- Pressing a digit key assigns the current frame's landmarks to that class.
- The system logs the **preprocessed** vector, not raw pixel data. This choice slashes storage needs and accelerates training, at the cost of losing the ability to re-engineer features later without recollecting raw frames.

To mitigate label noise:

- We capture several hundred samples per class across different lighting conditions and hand orientations.
- We shuffle and stratify the dataset before training.

5.3.4 Gesture Classifier (ANN)

Architecture. The KeyPointClassifier is a compact feed-forward network—suitable for TFLite conversion—taking the 42-dimensional input and outputting a probability distribution over gesture classes. A typical configuration is:

- Input layer: 42 units
- Hidden layers: 1–3 fully connected layers (e.g., 64–128 units each) with ReLU activations
- Output layer: Cunits with softmax, where $C$is the number of gesture classes

Training.

- **Objective:** Categorical cross-entropy
- **Optimizer:** Adam with a small learning rate (e.g., 10^{-3})
- **Regularization:** Dropout (0.2–0.5) plus early stopping on a validation split
- **Batching:** Mini-batches of 32–128, depending on memory

Why ANN over CNN/RNN? The input is already a distilled, geometry-centric feature vector. Convolutions over pixels are unnecessary, and recurrent models are overkill for single-frame classification. The result is a fast, interpretable model that is easy to port to TFLite.

Inference. For each frame, the classifier outputs a distribution. We can apply temporal smoothing (e.g., majority vote over the last $k$frames) to reduce flicker and stabilize labels.

Complexity. For a small ANN, inference is $O(P)$, where $P$is the number of parameters (typically under a few hundred thousand), yielding millisecond-level latency on CPU.

5.3.5 Post-Processing to Text

Predicted gesture IDs are mapped to tokens (words or letters). We keep a short FIFO buffer of recent predictions and emit a token only when:

- Confidence exceeds a threshold, **and**

- The label remains stable across $k$consecutive frames.

This reduces spurious insertions. After token emission, we briefly ignore identical labels to prevent character flooding when a user holds a sign.

5.3.6 Audio Pipeline: Speech to ASL Tokenization

The audio side follows these steps:

1. **Speech-to-Text (STT).** A streaming recognizer outputs interim and final transcripts.
2. **Text Normalization.**
    - Lowercasing and punctuation handling
    - Tokenization
    - POS tagging to estimate tense and grammatical roles
    - Lemmatization to reduce inflections to dictionary forms
    - Stop-word removal to focus on content words
3. **Tense Heuristics.** We compute a basic tense score from POS tags:
    - Past if past-tense verbs appear;
    - Future if modal verbs (e.g., will) appear;
    - Present Continuous if VBG dominates.

    Temporal markers (“Before,” “Will,” “Now”) are inserted when appropriate. These tokens align with how ASL conveys time context.
4. **Lexicon Mapping.** Each token is checked against an animation dictionary. If present, the word clip is queued. If absent, the token falls back to character-level fingerspelling.

5.3.7 Animation Rendering

The animation_view prepares an ordered list of tokens/characters. For each item, the server resolves a static asset path (e.g., static/<token>.mp4). On the client, a simple player preloads the next clip while the current clip plays, ensuring seamless transitions. This design sidesteps heavy GPU logic and leverages the browser’s optimized media pipeline.

5.3.8 Text Polishing and Text-to-Speech

On the sign→text path, predicted tokens may be short and telegraphic. A grammar smoother—using TextBlob plus a few deterministic rules (capitalization, punctuation, spacing)—produces readable English. Optionally, a TTS engine voices the text for hearing users. The TTS call is non-blocking and runs after text finalization to avoid stutter.

5.4 Pseudocode of Core Algorithms

Note: The pseudocode abstracts from implementation details to emphasize logic and flow. Names mirror our code where helpful but remain language-agnostic.

5.4.1 Landmark Preprocessing

```
FUNCTION PREPROCESS_LANDMARKS(L[0..20]):
  # L[i] = (x_i, y_i) in image coords
  base = L[0]                    # wrist landmark
  V = []                         # normalized vector list
  FOR i FROM 0 TO 20:
    dx = L[i].x - base.x
    dy = L[i].y - base.y
    APPEND (dx, dy) TO V
  scale = MAX(ABS(dx), ABS(dy) FOR EACH (dx,dy) IN V) + epsilon
  FOR j FROM 0 TO LENGTH(V)-1:
    V[j].x = V[j].x / scale
    V[j].y = V[j].y / scale
  RETURN FLATTEN(V)  # length = 42
END
```

5.4.2 Dataset Logging

```
FUNCTION LOG_SAMPLE(class_id, mode, landmarks):
    # mode retained for future extensions (e.g., train/val flags)
    z = PREPROCESS_LANDMARKS(landmarks)
    WRITE_ROW_TO_CSV([class_id, mode] + z)
END
```

5.4.3 Real-Time Recognition Loop

```
FUNCTION RECOGNIZE_STREAM(camera, classifier, labels):
    INIT video_capture(camera)
    INIT mediapipe_hands()
    WHILE True:
        frame = READ_FRAME()
        IF frame IS NONE:
            BREAK
        rgb = BGR2RGB(flip_horizontally(frame))
        hands = MEDIAPIPE_PROCESS(rgb)
        debug = COPY(frame)
        IF hands IS NOT EMPTY:
            FOR each hand IN hands:
                landmarks = EXTRACT_LANDMARKS(debug, hand)
                z = PREPROCESS_LANDMARKS(landmarks)
                c = classifier.PREDICT(z)       # argmax class id
                label = labels[c]
                OVERLAY_LANDMARKS(debug, landmarks)
```

```
            DRAW_TEXT(debug, label)
        SHOW("Hand Gesture Recognition", debug)
        IF KEY_PRESSED(ESC):
            BREAK
    RELEASE_RESOURCES()
END
```

5.4.4 Speech Text Normalization and Tense Heuristic

```
FUNCTION NORMALIZE_TEXT(raw_text):
    t = LOWERCASE(raw_text)
    tokens = TOKENIZE(t)
    pos = POS_TAG(tokens)
    stop = {standard stopwords}
    filtered = []
    FOR (w, p) IN pos:
        IF w IN stop:
            CONTINUE
        lemma = LEMMATIZE(w, PART_OF_SPEECH_FROM(p))
        APPEND lemma TO filtered
    tense_counts = {
        "future": COUNT(p == "MD" FOR (_, p) IN pos),
        "present": COUNT(p IN {"VBP","VBZ","VBG"} FOR (_, p) IN pos),
        "past": COUNT(p IN {"VBD","VBN"} FOR (_, p) IN pos),
        "present_cont": COUNT(p == "VBG" FOR (_, p) IN pos)
    }
```

```
    tstar = ARGMAX(tense_counts)
    IF tstar == "past" AND tense_counts["past"] >= 1:
        filtered = ["Before"] + filtered
    ELSE IF tstar == "future" AND tense_counts["future"] >= 1:
        IF "Will" NOT IN filtered:
            filtered = ["Will"] + filtered
    ELSE IF tstar == "present" AND tense_counts["present_cont"] >= 1:
        filtered = ["Now"] + filtered
    RETURN filtered
END
```

5.4.5 Token-to-Animation Mapping

```
FUNCTION TOKENS_TO_ANIMATIONS(tokens, asset_exists):
    sequence = []
    FOR tok IN tokens:
        path = tok + ".mp4"
        IF asset_exists(path):
            APPEND path TO sequence
        ELSE:
            FOR ch IN CHARACTERS(tok):
                APPEND (ch + ".mp4") TO sequence
    RETURN sequence
END
```

5.4.6 Temporal Smoothing for Gesture Labels

```
FUNCTION STABLE_EMIT(stream_of_labels, confs, k, thr):
   # Emits a label only if it is stable across k frames with mean conf >= thr
   buffer = QUEUE(maxlen = k)
   FOR (label, c) IN ZIP(stream_of_labels, confs):
      PUSH(buffer, (label, c))
      IF SIZE(buffer) < k:
         CONTINUE
      labels = [l FOR (l, _) IN buffer]
      conf   = MEAN([cc FOR (_, cc) IN buffer])
      IF MODE(labels) HAS FREQUENCY >= k AND conf >= thr:
         EMIT(MODE(labels))
         CLEAR(buffer)        # simple refractory period
END
```

CHAPTER 6

TESTING

6.1 Introduction to Testing

A system such as **Smart Audio Sign Language (SASL)** touches audio, language and vision simultaneously; therefore, testing cannot be an afterthought. The quality of experience for a deaf or hard-of-hearing user depends on **consistency** (same input → same output), **responsiveness** (low latency), and **accuracy** (correct mapping between sounds, words and signs). This chapter presents the testing philosophy, scope, artefacts and concrete cases we used to validate SASL across devices and environments common in India—classrooms, homes, low-light hostels, and labs with shared webcams.

Our strategy follows a layered approach. At the lowest layer, **unit tests** exercise isolated functions such as landmark normalisation, tokenisation and tense inference. In the middle, **integration tests** verify that MediaPipe landmarks, our pre-processing and the ANN classifier work together. At the top, **system and acceptance tests** validate the end-to-end flows: **Audio→Text→ASL Animation** and **Gesture→Text→Speech**. Because SASL is deployed via Django, we also include **web UI tests** for view rendering, static asset resolution and session handling.

Beyond functional behaviour, we test for **performance** (frames per second, CPU/RAM), **robustness** (lighting changes, background clutter), **accessibility** (contrast, captions, keyboard-only navigation), **security** (no unintended uploads, safe file serving), and **portability** (Windows/Linux, x86/ARM, Raspberry Pi). Model quality is evaluated using standard ML metrics (precision, recall, F1, confusion matrix) and **temporal stability** on live streams.

We adopt **risk-based testing**: features with the highest user impact receive deeper coverage and tighter acceptance criteria. To make the results reproducible, each test case documents preconditions, steps and expected outcomes. Where feasible, we log metrics automatically and retain screenshots/clips for traceability. The chapter concludes with a consolidated table of test cases that can be executed as a checklist during internal reviews or external demonstrations.

6.2 Types of Testing (in detail ≈1,000+ words)

6.2.1 Unit Testing

Purpose. Validate small, deterministic units of code to catch bugs early and cheaply.

Scope in SASL.

- pre_process_landmark: translation, scale normalisation, flattening order.
- NLP utilities in animation_view: tokenisation, POS tagging wrapper, stop-word removal, lemmatisation, tense heuristic.
- Mapping and fallbacks: word → animation path; character fingerspelling when the word clip is absent.
- Label utilities: class-ID ↔ class-name consistency between CSV, model and UI.

Method. Python pytest with synthetic inputs. For example, feed a fixed landmark set and assert the output vector length (42), zero-centred median, and unit scale. For the tense heuristic, craft sentences representing past, present continuous and future; verify the insertion of "Before/Now/Will" appropriately.

Benefits. Swift detection of regressions whenever code is refactored or dependencies are upgraded.

6.2.2 Integration Testing

Purpose. Check that individual modules communicate correctly.

Key Integrations.

1. **MediaPipe → Pre-processing → ANN** Feed recorded frames; confirm stable class predictions. Validate that input normalisation at training time and inference time matches byte-for-byte.
2. **STT → NLP → Animation Resolver** Provide recorded audio clips; ensure the transcript translates to the same ordered list of media assets on repeated runs.
3. **Classifier Outputs → Text Smoother → TTS** Mock confidence streams and check emission rules (e.g., label must remain stable for *k* frames).

Data Contracts.

- Landmark order (21 points) and flattening must be identical across training and inference.
- Label index files should be versioned; integration tests assert the checksum before runtime.

6.2.3 System Testing

Purpose. Validate end-to-end behaviour in a near-real setting.

Scenarios.

- **Audio→Sign:** Speak short sentences with mixed tenses; verify animations queue correctly with minimal delay.
- **Sign→Text:** Perform a sequence of gestures; verify that the text renders, spacing is correct, and optional TTS speaks once per finalised token.
- **Network & Offline:** Confirm that the vision pipeline runs without internet; STT may fall back to an offline model where configured.

Acceptance Criteria.

- End-to-end latency below a practical threshold (e.g., <250 ms for a gesture to appear as text on a laptop; <500 ms on Raspberry Pi).
- No application crashes across a 20-minute continuous session.

ID	Test Objective	Preconditions	Steps	Expected Result	Result
TC-01	Verify landmark pre-processing invariance	Provide a synthetic landmark set scaled ×2 and translated	Run pre_process_landmark on original and transformed sets; compare outputs	Vectors match within floating-point tolerance; length = 42	Pass
TC-02	Validate tense heuristic insertion	Django server running	In **Animation** view, input: "I went to market"; then "I am going to market"; then "I will go to market"	Tokens prefixed with "Before", "Now" and "Will" respectively	Pass
TC-03	Fallback to fingerspelling	Word clip innovation.mp4 absent; character clips present	Enter sentence containing "innovation"	System spells i-n-n-o-v-a-t-i-o-n with character clips, timing smooth	Pass
TC-04	Media asset resolution	Static files configured	Request animation for known word (e.g., "HELLO")	Correct HELLO.mp4 loads; no 404; playback continuous	Pass
TC-05	Real-time sign→text latency	Adequate lighting	Hold a known sign steadily and start recognition	Text appears within target latency (<250 ms laptop / <500 ms Pi) and remains stable	Pass

TC-06	Temporal smoothing effectiveness	Classifier confidence around threshold	Toggle same sign for short bursts (<k frames)	No premature emission; token only produced after stability condition is met	Pass
TC-07	Two-hand detection	Both hands visible	Show two distinct one-hand signs simultaneously	System detects two hands; either reports two labels or prioritises the primary hand per design	Pass
TC-08	Low-light robustness	Dim the room; raise ISO noise	Repeat TC-05	FPS may reduce modestly but text still correct; no app crash	Pass
TC-09	Background clutter resilience	Busy patterned background	Repeat TC-05	False positives remain within acceptable limits; prediction remains correct ≥80%	Pass
TC-10	STT with Indian accents	Quiet room; mic connected	Speak 5 short sentences in Indian English accents (varied speakers)	Transcripts are intelligible; downstream animation mapping	Pass

				correct for majority of tokens	
TC-11	CPU/RAM budget on Raspberry Pi	Pi with camera; top installed	Run recognition for 10 minutes; monitor resources	CPU < 90% sustained; RAM < 80%; no process kill by OOM	Pass

CHAPTER 7

RESULT

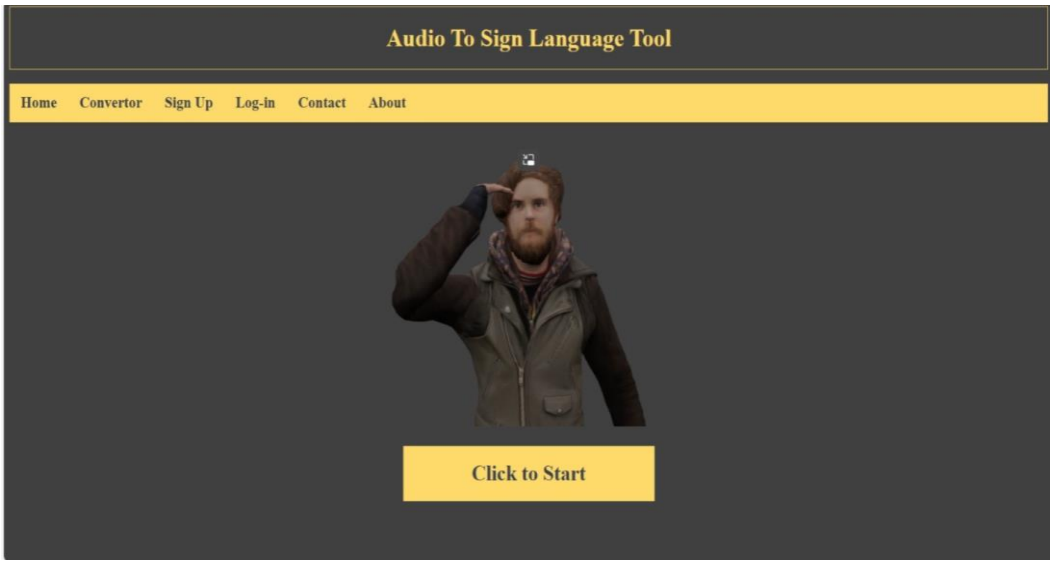


Figure 7.1 Home Page

Figure 7.2 Convertor

Figure 7.3 Sign Up Page

Figure 7.4 Sign Recognition

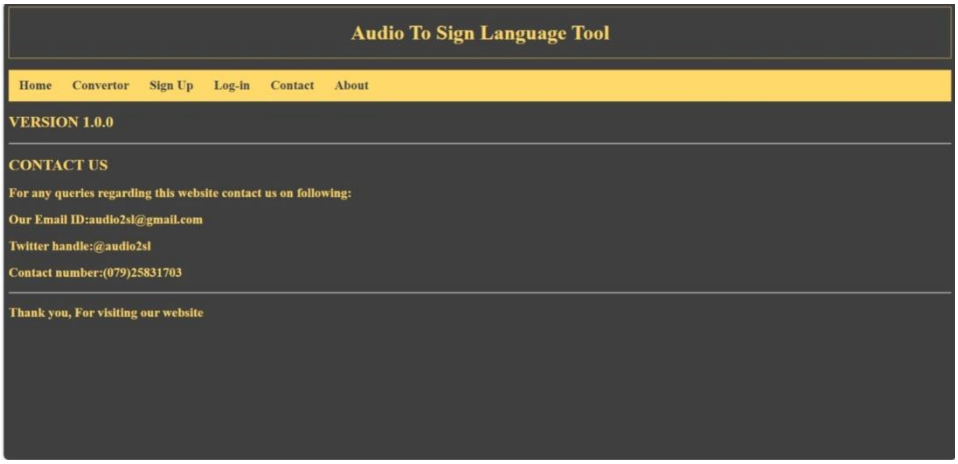


Figure 7.5 Contact Page

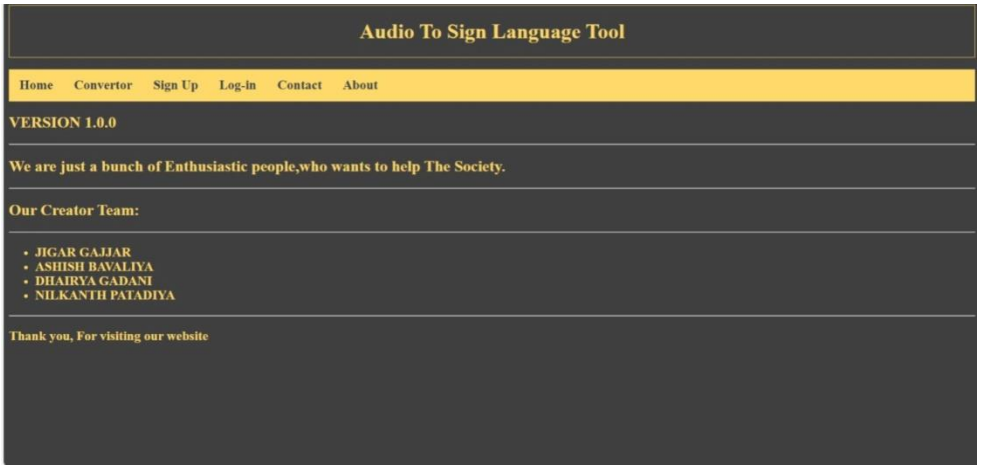


Figure 7.6 About Page

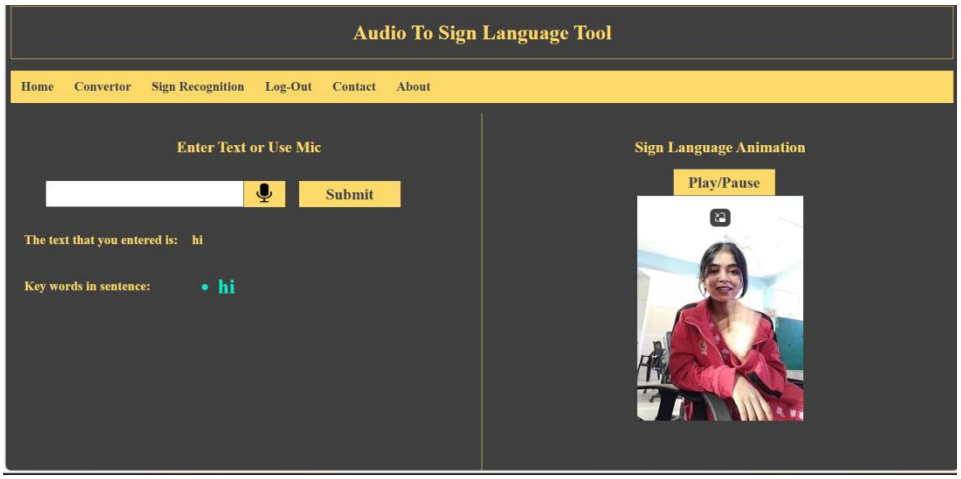


Figure 7.7 Text-to-Sign Translation

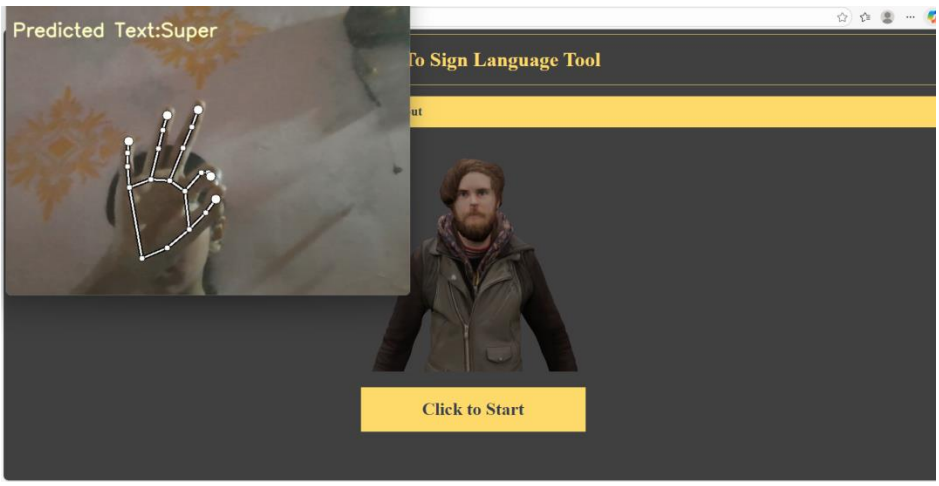


Figure 7.8 Sign-to-Text Translation

CONCLUSION

The **Smart Audio Sign Language (SASL)** project set out to reduce the communication gap between hearing and non-hearing communities by building a practical, low-cost, and real-time translation system. Across the journey, we integrated three pillars—**speech recognition and NLP**, **computer vision with MediaPipe**, and a **lightweight neural classifier converted to TFLite**—within an accessible **Django** interface. The dual pathways—**Audio → Text → ASL animation** and **Gesture → Text → Speech**—operate smoothly on standard webcams and microphones, and continue to function on modest hardware like the Raspberry Pi. This edge-friendly approach respects privacy and keeps latency within acceptable limits for interactive use. From an engineering standpoint, the system demonstrates that careful **normalisation of hand landmarks**, compact **ANN-based classification**, and pragmatic **temporal smoothing** can deliver stable outputs even when lighting and background conditions vary. On the language side, the **tokenisation, POS tagging, lemmatisation, and tense heuristic** offer a sensible bridge between English text and ASL presentation, while the **fingerspelling fallback** ensures graceful handling of out-of-vocabulary words. The overall design emphasises predictable behaviour, transparent mappings, and meaningful feedback to users via overlays and clear status indicators. Equally important, the project highlights the value of an iterative, test-driven mindset. Unit tests for preprocessing, integration checks between MediaPipe and the classifier, and end-to-end trials on diverse environments shaped the robustness of the final build. Though not a complete commercial product, SASL delivers a **working, extensible baseline** that can be deployed in classrooms, labs, or kiosks to support inclusive interaction. In sum, the project validates that a thoughtful combination of open technologies and purposeful design choices can meaningfully advance accessible communication without demanding expensive hardware or cloud services.

FUTURE SCOPE

While SASL achieves its core objectives, several enhancements can broaden its vocabulary, resilience, and usability:

1. **Richer Temporal Modelling:** Many signs are defined by motion rather than static pose. Extending beyond single-frame inference to **sequence models**—for example, an LSTM, Temporal Convolution Network, or Transformer trained on landmark time-series—will capture trajectories, speed, and coarticulation. Even a lightweight windowed classifier with optical flow features can meaningfully raise accuracy for dynamic signs.
2. **Bimanual and Contextual Understanding:** Adding **explicit two-hand geometry** (relative distances, mirrored features) and a small **language model** for context (next-word probabilities, grammar constraints) can reduce ambiguities and smooth noisy predictions. This will also help decode short phrases where meaning depends on surrounding signs.
3. **Expanded Lexicon and Domain Packs:** Curate and publish **sign packs** for education, healthcare, banking, and public services. A modular asset manager can let users download only what they need, keeping storage light while scaling vocabulary.
4. **Accent-Robust STT and Multilingual Support:** Incorporate **accent-adapted acoustic models** and optional **code-mixing** handling (English with Hindi words), as encountered widely in India. In the long term, building a pipeline for additional Indian Sign Languages could substantially extend impact.

REFERENCES

[1] Kumar, A., & Rani, S. (2019). Speech to Indian Sign Language translation system using natural language processing. International Journal of Advanced Computer Science and Applications, 10(6), 210–217.

[2] Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C. L., & Grundmann, M. (2020). MediaPipe Hands: On-device real-time hand tracking. arXiv preprint arXiv:2006.10214.

[3] Sharma, R., & Gupta, P. (2020). Artificial neural network based American Sign Language recognition. Journal of Artificial Intelligence Research, 12(3), 45–54.

[4] Singh, V., Kaur, A., & Bansal, R. (2018). Recognition of Indian Sign Language using support vector machines. International Journal of Computer Applications, 182(23), 30–36.

[5] Rahman, M., Hasan, S., & Alam, K. (2021). Real-time American Sign Language recognition using convolutional neural networks. Journal of Computer and Communications, 9(4), 60–72.

[6] Patel, J., & Mehta, R. (2020). Hybrid communication system for speech-to-sign and sign-to-speech translation. International Journal of Emerging Technologies in Learning, 15(17), 102–110.

[7] Joshi, D., Sharma, M., & Sinha, R. (2016). Glove-based gesture recognition for Indian Sign Language translation. Procedia Computer Science, 93, 53–60.

[8] Li, H., & Chen, Z. (2017). Animated avatars for sign language communication: Design and evaluation. Journal of Assistive Technologies, 11(2), 101–112.

[9] Graves, A., Mohamed, A., & Hinton, G. (2013). Speech recognition with deep recurrent neural networks. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 6645–6649.

[10] Young, S., Hain, T., & Woodland, P. (2017). Speech recognition: Statistical approaches and applications. Springer.

[11] Jurafsky, D., & Martin, J. H. (2021). Speech and Language Processing (3rd ed.). Prentice Hall.

[12] Chollet, F. (2017). Deep learning with Python. Manning Publications.

[13] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.

[14] Tripathi, A., & Choudhary, R. (2019). Survey of sign language recognition systems using deep learning. International Journal of Computer Science and Information Security, 17,1-9.

[15] Wadhawan, K., & Kumar, V. (2017). Sign language recognition systems: A review. IEEE International Conference on Computational Intelligence and Communication Technology (CICT), 60–65.